Լաբորատոր աշխատանք 6

1. Խնդրի նպատակը

Մշակել օբյեկտ կողմնորոշված ծրագիր (C++ լեզվով)՝ ուսանողների առաջադիմության, գնահատականների և հաճախելիության հաշվառման, վերլուծության ու վարկանիշային ցուցակների կազմման համար։

2. Հիմնական պահանջները

- **Դասի ստեղծում (Student)**. ունենալ private դաշտեր (id, name, midterm, labPoints, exam, absences), public կոնստրուկտոր, setter/getter մեթոդներ և մուտքային տվյալների վավերացում։
- **Սահմանափակումների կիրառում**.
    - Միավորների առավելագույն շեմեր՝ միջանկյալ (20), լաբորատոր (30), քննություն (50)։
    - Բացակայությունների կանոն՝ 6-ից ավելի բացակայության դեպքում ուսանողը չի կարող մասնակցել քննությանը (քննության միավորը զրոյացվում է)։
- **Վերլուծություն**. հաշվել վերջնական միավորը, տառային գնահատականը, գտնել դասընթացը չհաղթահարածներին և հաշվել յուրաքանչյուր բաղադրիչի միջին միավորը։
- **Վարկանիշային ցուցակ**. դասավորել ուսանողներին ըստ վերջնական միավորի նվազման, իսկ միավորների հավասարության դեպքում՝ ըստ լաբորատոր աշխատանքների միավորների։

3. Տեսական հիմքեր

- **Օբյեկտ կողմնորոշված ծրագրավորում (OOP)**.
    - *Encapsulation (Գաղտնագրում/Պարկուճավորում)*. տվյալների պաշտպանություն private մուտքի սահմանափակմամբ և դրանց կառավարում public մեթոդների (setter/getter) միջոցով։
- **Տվյալների կառուցվածքներ (Data Structures)**.
    - Օբյեկտների զանգվածի կամ std::vector-ի կիրառում՝ բազմաթիվ ուսանողների տվյալները միասին պահելու և մշակելու համար։

- **Ալգորիթմեր և Սորտավորում (Sorting)**.
    - C++ STL-ի std::sort ֆունկցիայի և բազմապայմանանի համեմատման (custom comparator) կիրառում՝ ուսանողներին ըստ որոշակի չափանիշների դասավորելու համար։
    - #include <iostream>
    - #include <vector>
    - #include <string>
    - #include <iomanip>
    - #include <algorithm>
    -
    - class Student {
    - private:
    - int id;
    - std::string name;
    - double midterm; // Առավելագույնը 20 միավոր
    - double labPoints; // Առավելագույնը 30 միավոր
    - double exam; // Առավելագույնը 50 միավոր
    - int absences; // Բացակայությունների քանակ
    -
    - // Սահմանային արժեքներ (կոնստանտներ)
    - static constexpr double MAX_MIDTERM = 20.0;
    - static constexpr double MAX_LAB = 30.0;
    - static constexpr double MAX_EXAM = 50.0;
    - static constexpr int MAX_ABSENCES_FOR_EXAM = 6;
    -
    - public:
    - // Կոնստրուկտոր
    - Student(int studentId, std::string studentName, double m, double l, double e, int abs) {
    - id = studentId;
    - name = studentName;
    - setMidterm(m);
    - setLabPoints(l);
    - setExam(e);

```
        setAbsences(abs);
    }

    // Setter-ներ վավերացմամբ
    void setMidterm(double m) {
        midterm = (m >= 0 && m <= MAX_MIDTERM) ? m : 0;
    }

    void setLabPoints(double l) {
        labPoints = (l >= 0 && l <= MAX_LAB) ? l : 0;
    }

    void setExam(double e) {
        // Եթե բացակայությունները 6-ից ավել են, քննությանը մասնակցել չի կարող (միավորը = 0)
        if (absences > MAX_ABSENCES_FOR_EXAM) {
            exam = 0;
        } else {
            exam = (e >= 0 && e <= MAX_EXAM) ? e : 0;
        }
    }

    void setAbsences(int abs) {
        absences = (abs >= 0) ? abs : 0;
        // Եթե բացակայությունները դառնում են > 6, թարմացնում ենք քննության միավորը
        if (absences > MAX_ABSENCES_FOR_EXAM) {
            exam = 0;
        }
    }

    // Getter-ներ
    int getId() const { return id; }
    std::string getName() const { return name; }
    double getMidterm() const { return midterm; }
```

```
    double getLabPoints() const { return labPoints; }
    double getExam() const { return exam; }
    int getAbsences() const { return absences; }

    // Վերջնական միավորի հաշվարկ (Միջանկյալ + Լաբորատոր + Քննություն)
    double getFinalScore() const {
      return midterm + labPoints + exam;
    }

    // Տառային գնահատականի և անցողիկության ստուգում
    std::string getLetterGrade() const {
      double finalScore = getFinalScore();
      if (finalScore >= 90) return "A";
      if (finalScore >= 80) return "B";
      if (finalScore >= 70) return "C";
      if (finalScore >= 60) return "D";
      return "F";
    }
    bool hasPassed() const {
      return getFinalScore() >= 60 && absences <= MAX_ABSENCES_FOR_EXAM;
    }
    //  տեղեկատվության արտածում
    void printInfo() const {
      std::cout << std::left << std::setw(5) << id
            << std::setw(20) << name
            << std::setw(10) << midterm
            << std::setw(10) << labPoints
            << std::setw(10) << exam
            << std::setw(12) << absences
            << std::setw(10) << getFinalScore()
            << std::setw(8)  << getLetterGrade() << "\n";
    }
  };
```

```
o
o   // Համեմատման ֆունկցիա վարկանիշային ցուցակի համար (Սորտավորում)
o   bool compareStudents(const Student& a, const Student& b) {
o     if (a.getFinalScore() != b.getFinalScore()) {
o       return a.getFinalScore() > b.getFinalScore(); // Նվազման կարգով ըստ վերջնական միավորի
o     }
o     return a.getLabPoints() > b.getLabPoints(); // Հավասարության դեպքում՝ ըստ լաբորատոր միավորի
o   }
o   int main() {
o     // Ուղղորդված տողային կոդավորում (UTF-8 աջակցության համար)
o     std::locale::global(std::locale("en_US.UTF-8"));
o
o     // 1. Ստեղծել 15 ուսանողների զանգված (vector)
o     std::vector<Student> students = {
o       Student(1,  "Արմեն Հակոբյան",  18, 28, 45, 2),
o       Student(2,  "Աննա Պետրոսյան",   15, 22, 35, 7), // Բացակայություն > 6 (քննության միավորը կզրոյացվի)
o       Student(3,  "Գևորգ Սարգսյան",  20, 30, 48, 1),
o       Student(4,  "Մարիամ Վարդանյան", 12, 18, 25, 4),
o       Student(5,  "Դավիթ Մկրտչյան",  19, 27, 42, 0),
o       Student(6,  "Լիլիթ Կարապետյան", 10, 15, 20, 8), // Բացակայություն > 6
o       Student(7,  "Տիգրան Խաչատրյան", 17, 25, 38, 3),
o       Student(8,  "Սոնա Հովհաննիսյան", 14, 20, 30, 5),
o       Student(9,  "Վահե Գրիգորյան",   8,  12, 15, 6),
o       Student(10, "Նարե Ավագյան",     20, 29, 46, 1),
o       Student(11, "Կարեն Մանուկյան",  16, 24, 32, 2),
o       Student(12, "Էլեն Հարությունյան", 13, 19, 28, 4),
o       Student(13, "Հայկ Բաղդասարյան", 19, 26, 40, 3),
o       Student(14, "Արփինե Միքայելյան", 11, 16, 22, 5),
o       Student(15, "Սմբատ Ցավրումյան", 5,  10, 10, 9)  // Բացակայություն > 6
o     };
o
```

```
    // Աղյուսակի վերնագիր
    std::cout << "--- ՀԱՄԱԼՍԱՐԱՆԻ ՈՒՍԱՆՈՂՆԵՐԻ ՑՈՒՑԱԿ ---\n";
    std::cout << std::left << std::setw(5) << "ID"
              << std::setw(20) << "Անուն Ազգանուն"
              << std::setw(10) << "Միջանկ."
              << std::setw(10) << "Լաբ."
              << std::setw(10) << "Քննություն"
              << std::setw(12) << "Բացակա"
              << std::setw(10) << "Ընդամենը"
              << std::setw(8)  << "Գնահ." << "\n";
    std::cout << std::string(85, '-') << "\n";

    for (const auto& student : students) {
        student.printInfo();
    }
    std::cout << "\n";
    // 2. Գտնել դասընթացը չհաղթահարած ուսանողներին
    std::cout << "--- ԴԱՍԸՆԹԱՑԸ ՉՀԱՂԹԱՀԱՐԱԾ (ԿԱՄ ՔՆՆՎԵԼՈՒ ՀԱՄԱՐ ԱՆՈՒՂԵԼԻ ԲԱՑԱԿԱՅՈՒԹՅՈՒՆ ՈՒՆԵՑՈՂ) ՈՒՍԱՆՈՂՆԵՐ ---\n";
    bool anyFailed = false;
    for (const auto& student : students) {
        if (!student.hasPassed()) {
            std::cout << "ID: " << student.getId() << " | Անուն: " << student.getName()
                      << " | Վերջնական միավոր: " << student.getFinalScore()
                      << " | Բացակայություններ: " << student.getAbsences() << "\n";
            anyFailed = true;
        }
    }
    if (!anyFailed) std::cout << "Բոլոր ուսանողները հաղթահարել են դասընթացը:\n";
    std::cout << "\n";
    // 3. Հաշվել յուրաքանչյուր բաղադրիչի միջին միավորը
    double totalMidterm = 0, totalLab = 0, totalExam = 0, totalFinal = 0;
    int count = students.size();
```

```
    for (const auto& student : students) {
        totalMidterm += student.getMidterm();
        totalLab += student.getLabPoints();
        totalExam += student.getExam();
        totalFinal += student.getFinalScore();
    }
    std::cout << "--- ԲԱՂԱԴՐԻՉՆԵՐԻ ՄԻՋԻՆ ՄԻԱՎՈՐՆԵՐ ---\n";
    std::cout << "Միջանկյալ ստուգման միջինը: " << (totalMidterm / count) << " / 20\n";
    std::cout << "Լաբորատոր աշխատանքների միջինը: " << (totalLab / count) << " / 30\n";
    std::cout << "Քննության միջինը: " << (totalExam / count) << " / 50\n";
    std::cout << "Ընդհանուր վերջնական միջինը: " << (totalFinal / count) << " / 100\n\n";

    // 4. Վարկանիշային ցուցակ (Սորտավորում ըստ վերջնական միավորի, հավասարության դեպքում՝ ըստ լաբորատորի)
    std::sort(students.begin(), students.end(), compareStudents);

    std::cout << "--- ՎԱՐԿԱՆԻՇԱՅԻՆ ՑՈՒՑԱԿ (ՍՈՐՏԱՎՈՐՎԱԾ) ---\n";
    std::cout << std::left << std::setw(5) << "Տեղ"
              << std::setw(20) << "Անուն Ազգանուն"
              << std::setw(10) << "Վերջն. միավոր"
              << std::setw(10) << "Լաբ. միավոր" << "\n";
    std::cout << std::string(50, '-') << "\n";
    int rank = 1;
    for (const auto& student : students) {
        std::cout << std::left << std::setw(5) << rank++
                  << std::setw(20) << student.getName()
                  << std::setw(10) << student.getFinalScore()
                  << std::setw(10) << student.getLabPoints() << "\n";
    }

    return 0;
}
```